Ensemble Methods

TP – Risk certificates for ensemble methods – Solutions

M2 Computer Science – MALIA

Guillaume Metzler
Institut de Communication (ICOM)
Université de Lyon, Université Lumière Lyon 2
Laboratoire ERIC UR 3083, Lyon, France

guillaume.metzler@univ-lyon2.fr

Abstract. In this practical session you will compute PAC-Bayesian risk certificates for random forests, Gibbs posteriors, boosted ensembles and linear classifiers on the datasets of the course, learn the weights of majority votes by minimizing these certificates, use them for model selection, and compare them with the classical hold-out certificate. The session relies on notebooks 1 to 5 and on the module `pbtools.py`. The work is organized in **four sessions of three hours**, each starting on a new page with its goals, the sections of the lecture notes to reread, a suggested timing and the results to hand in; parts are numbered by session (Part 2.3 is the third part of Session 2). The sessions build on each other, but the parts inside a session are independent after its first part and can be split among the members of a group. Parts marked ($\star$) are longer. All the numbers of these solutions are produced by the script `tp_solutions.py` (also available as the notebook `tp_pacbayes_solutions.ipynb`), whose functions are named after the parts (one notebook per session); each solution gives the pseudo-code of the computation, the results and their discussion. Unless stated otherwise, $\delta = 0.05$, splits are stratified 70/30 train/test splits (5 splits for Parts 1.3, 2.1, 3.3 and 3.4, 3 splits elsewhere), and forests have 100 trees trained on bootstrap samples of size $m/2$; your numbers will differ slightly with other seeds.

Session	Theme	Parts
1	Certifying a classifier and a forest	warm-up on the kl, selection bias, OOB certificates, trees and bootstrap size
2	Self-bounding learning with forests	learning the weights, exact kl minimization, strength vs diversity, model selection
3	Gibbs posteriors and hard cases	temperature of a Gibbs posterior, hold-out vs self-bounding, imbalance, boosting
4	Linear classifiers and synthesis	PBGD and Monte Carlo certificates, data-dependent priors, synthesis, report

Session 1 – Certifying one classifier and a random forest

Goal. By the end of the session you should be able to compute a test-set bound by hand and with the kl inversion, understand selection bias, and compute the first-order, tandem and C-bound certificates of a random forest from its out-of-bag samples.
To reread before the session. Sections 2, 5 and 7.2 of the notes; notebooks 1 and 3.
Timing (3 hours).

	Part 1.1	20 min
	Part 1.2	50 min
	Part 1.3	60 min
	Part 1.4	50 min

To hand in at the end of the session. Your notebook, with the table of Part 1.3 on your 10–15 datasets and the table of Part 1.4.

Part 1.1 – Setting up

- Download the datasets (`datasets.zip`) and the loading script `prepdata.py` of the first practical session, and the module `pbtools.py`. The function `data_recovery(name)` returns a pair `(X, y)` with labels in $\{0, 1\}$: convert them to $\{-1, +1\}$ and standardize the features.
- Read `pbtools.py`: every function is short. It provides the binary kl and its inversions (`kl_bin`, `kl_inv_upper`, `kl_inv_lower`), the classical bounds (`bound_seeger`, `bound_mcallester`, `bound_catoni`, `bound_lambda`), a random forest keeping track of its out-of-bag samples (`BaggedTrees` with `oob_losses`, `oob_tandem`, `oob_disagreement`), the majority-vote certificates (`fo_bound`, `tnd_bound`, `cbound_pb`), two optimizers (`optimize_fo`, `optimize_tnd`) and the gradient of the kl-form certificates (`certificate_and_grad`).
- Use **at least 10 datasets**, balanced and imbalanced, for instance `australian`, `balance`, `bupa`, `german`, `heart`, `iono`, `pima`, `sonar`, `wdbc`, `spambase` (balanced group) and `abalone8`, `pageblocks`, `satimage`, `segmentation`, `yeast3` (imbalanced group).

> **Good practice.** Fix all the random seeds, and write one function per experiment that returns a table: you will rerun them many times. Before computing a certificate, write down on paper *which examples* each voter has seen: this single question avoids most of the mistakes of this session.

Solution

All experiments use the same loading helper, which standardizes the features and maps the labels to $\{-1, +1\}$ (function `load`). The sizes range from 208 (`sonar`) to 6435 (`satimage`) examples, and the proportion of positive examples from 2% to 48%. Keep these sizes in mind: a PAC-Bayesian certificate is computed with the number of examples on which the voters are evaluated, and with only a few hundred examples it cannot be tight.

Part 1.2 – Warm-up: inverting the kl and certifying one classifier (Section 2)

1. Implement the upper inversion $\overline{\mathrm{kl}}^{-1}(\hat{q}, \varepsilon)$ by bisection (Algorithm 1 of the notes) and check it against `kl_inv_upper`. How many iterations are needed for a precision of 10^{-9}?
2. For $m \in \{100, 1000\}$, $\delta = 0.05$, $\varepsilon = \ln(1/\delta)/m$ and $\hat{q} \in \{0, 0.01, 0.05, 0.1, 0.3, 0.5\}$, compare three upper confidence bounds on the risk of a *fixed* classifier: Hoeffding $\hat{q} + \sqrt{\varepsilon/2}$, the kl inversion, and the refined Pinsker relaxation $\hat{q} + \sqrt{2\hat{q}\varepsilon} + 2\varepsilon$. When does the kl make a difference?
3. On `australian`, split the data in two halves. Learn a decision tree of depth 3 on the first half and compute the Hoeffding and kl test-set bounds of its risk on the second half.
4. Now learn trees of depths $1, \ldots, 10$ on the first half, keep the one with the smallest error on the second half, and compute its kl bound as if it had been fixed in advance. Why is this "certificate" not valid? Correct it.

Solution

1. The bisection keeps an interval $[a, b]$ that contains $\overline{\mathrm{kl}}^{-1}(\hat{q}, \varepsilon)$, i.e. such that $\mathrm{kl}(\hat{q}\|a) \leq \varepsilon < \mathrm{kl}(\hat{q}\|b)$; since $p \mapsto \mathrm{kl}(\hat{q}\|p)$ is increasing on $[\hat{q}, 1]$, the midpoint tells on which side the answer lies.

Algorithm 1: `kl_inv_bisection`$(\hat{q}, \varepsilon, \tau)$

$a \leftarrow \hat{q}$; $b \leftarrow 1$;
while $b - a > \tau$ **do**
  $c \leftarrow (a+b)/2$;
  if $\mathrm{kl}(\hat{q}\|c) > \varepsilon$ **then** $b \leftarrow c$ **else** $a \leftarrow c$;
return b ; ▷ *upper end, hence a valid (conservative) bound*

Python: function `kl_inv_bisection, part1` of `tp_solutions.py`.
The interval length is divided by 2 at each step, starting from at most 1: $\lceil \log_2 10^9 \rceil = 30$ iterations give a precision of 10^{-9}. The implementation agrees with `kl_inv_upper` to 10^{-6} on all the values of question 2.
2. Results ($\delta = 0.05$):

m	$\hat{q}$	Hoeffding	kl	refined Pinsker
100	0.00	0.122	0.030	0.060
100	0.01	0.132	0.056	0.094
100	0.05	0.172	0.122	0.165
100	0.10	0.222	0.189	0.237
100	0.30	0.422	0.418	0.494
100	0.50	0.622	0.621	0.733
1000	0.00	0.039	0.003	0.006
1000	0.01	0.049	0.020	0.024
1000	0.05	0.089	0.069	0.073
1000	0.10	0.139	0.125	0.130
1000	0.30	0.339	0.336	0.348
1000	0.50	0.539	0.539	0.561

The Hoeffding width $\sqrt{\ln(1/\delta)/(2m)}$ does not depend on $\hat{q}$ (0.122 for $m = 100$, 0.039 for $m = 1000$). The kl interval is much narrower when $\hat{q}$ is small: for $\hat{q} = 0$ it is $1 - e^{-\varepsilon} \approx \varepsilon$, i.e. a width in $1/m$ instead of $1/\sqrt{m}$ (0.003 instead of 0.039 for $m = 1000$). When $\hat{q}$ approaches $1/2$ the two bounds coincide, as predicted by Pinsker's inequality, which is tight around $1/2$. The refined Pinsker bound captures the fast rate for small $\hat{q}$, but it becomes looser than Hoeffding's for large $\hat{q}$ (0.494 against 0.422 at $\hat{q} = 0.3$, $m = 100$); the kl is always the tightest of the three. The kl matters for good classifiers, which is precisely the case of interest.
3–4.

Algorithm 2: Test-set bound, and the same bound after a selection among K candidates

Split S into S_1 (training) and S_2 (test), $n = |S_2|$;
for $k = 1, \dots, K$ **do**
  learn h_k on S_1; $\hat{r}_k \leftarrow$ error of h_k on S_2;
$k^\star \leftarrow \arg\min_k \hat{r}_k$;
naive certificate $\leftarrow \overline{\mathrm{kl}}^{-1}\big(\hat{r}_{k^\star}, \ln(1/\delta)/n\big)$; ▷ *invalid: S_2 was used to select*
corrected certificate $\leftarrow \overline{\mathrm{kl}}^{-1}\big(\hat{r}_{k^\star}, \ln(K/\delta)/n\big)$; ▷ *Occam / union bound*

Python: function `part1` of `tp_solutions.py`.

n	error (depth 3)	Hoeffding	kl	selected depth	its error	naive kl	kl with $\delta/10$
345	0.142	0.208	0.192	8	0.139	0.189	0.207

With $n = 345$ test examples, the depth-3 tree has a test error of 0.142; the Hoeffding bound gives 0.208 and the kl bound 0.192. The selected depth (8) has been chosen because its error on the second half is the smallest: that half is no longer independent of the selected classifier, and the test-set bound, which holds for a classifier *fixed in advance*, does not apply (selection

bias, Figure 18 of the notes). The correction is the Occam bound with a uniform prior on the 10 candidates, i.e. a union bound: replacing δ by $\delta/10$ increases the certificate from 0.189 to 0.207. The price, $\ln 10 \approx 2.3$ nats, is small; the mistake would be to pay nothing.

Part 1.3 – Certificates of a standard random forest (Sections 5 and 7.2)

For each dataset, repeat 5 times: split the data into 70% training / 30% test (stratified) and learn a `BaggedTrees` forest of 100 trees on the training set with bootstrap samples of size $m/2$.

1. Compute the test risk of the uniform majority vote, the test Gibbs risk and disagreement, and the three OOB certificates (first-order, tandem, C-bound) of the uniform posterior, with $\delta = 0.05$. Report also the sizes n_1 (smallest OOB set) and n_2 (smallest intersection of two OOB sets).
2. Check that the certificates are always above the test risk. Which bound is the tightest, on which datasets? Relate your answer to the disagreement of the trees (Proposition 5.7) and to n_1, n_2.
3. Compare the test risk of `BaggedTrees` with `sklearn.ensemble.RandomForestClassifier` (full bootstrap). How much accuracy do we lose by using bootstrap samples of size $m/2$?

Solution

Algorithm 3: OOB certificates of a forest (uniform posterior π)

for $t = 1, \dots, n$ **do**
  draw a bootstrap sample S_t of size $m/2$; learn tree h_t on S_t; $T_t \leftarrow S \setminus S_t$;
  $\widehat{L}_t \leftarrow$ error of h_t on T_t;
for t, t' **do**
  $\widehat{L}_{tt'} \leftarrow$ frequency of joint errors of $h_t, h_{t'}$ on $T_t \cap T_{t'}$; $\widehat{D}_{tt'} \leftarrow$ frequency of disagreements on $T_t \cap T_{t'}$;
$n_1 \leftarrow \min_t |T_t|$; $n_2 \leftarrow \min_{t,t'} |T_t \cap T_{t'}|$;
FO $\leftarrow 2\,\overline{\mathrm{kl}}^{-1}(\pi^\top \widehat{L}, \frac{\ln(2\sqrt{n_1}/\delta)}{n_1})$; TND $\leftarrow 4\,\overline{\mathrm{kl}}^{-1}(\pi^\top \widehat{\mathbf{L}}\pi, \frac{\ln(2\sqrt{n_2}/\delta)}{n_2})$;
$\overline{r} \leftarrow \overline{\mathrm{kl}}^{-1}(\pi^\top \widehat{L}, \cdot)$, $\underline{d} \leftarrow \underline{\mathrm{kl}}^{-1}(\pi^\top \widehat{\mathbf{D}}\pi, \cdot)$, each with $\delta/2$;
C-bound $\leftarrow 1 - \frac{(1-2\overline{r})^2}{1-2\underline{d}}$; ▷ *Equation (5.5) of the notes*

Python: function `forest_stats, certificates, part2_3` of `tp_solutions.py`.
Mean values over the 5 splits (RF: `RandomForestClassifier`; MV, Gibbs, d: uniform forest on the test set; FO, TND, C: OOB certificates of the uniform vote):

dataset	RF	MV	Gibbs	d	n_1	n_2	FO	TND	C-bound
australian	0.137	0.141	0.234	0.258	280	154	0.649	0.819	0.839
balance	0.114	0.097	0.171	0.205	253	138	0.523	0.664	0.731
bupa	0.260	0.254	0.401	0.425	136	70	1.000	1.000	1.000
german	0.245	0.242	0.346	0.364	408	226	0.864	1.000	0.968
heart	0.153	0.158	0.280	0.323	106	54	0.934	1.000	0.996
iono	0.074	0.077	0.158	0.190	139	72	0.560	0.790	0.792
pima	0.250	0.249	0.331	0.327	310	171	0.848	1.000	0.965
sonar	0.194	0.216	0.351	0.394	81	39	1.000	1.000	1.000
wdbc	0.056	0.061	0.093	0.100	231	124	0.324	0.476	0.528
spambase	0.051	0.056	0.124	0.156	1919	1119	0.306	0.289	0.380
abalone8	0.142	0.141	0.211	0.205	1741	1017	0.502	0.591	0.639
pageblocks	0.023	0.021	0.042	0.048	2283	1339	0.124	0.150	0.190
satimage	0.059	0.062	0.100	0.111	2695	1577	0.248	0.271	0.329
segmentation	0.024	0.027	0.070	0.093	958	551	0.210	0.221	0.310
yeast3	0.050	0.046	0.086	0.101	612	346	0.274	0.352	0.424

1–2. All certificates are above the test risk, as they must be (a violation would happen with probability at most 5% per split). Their quality depends above all on the number of OOB examples. On the small datasets (`bupa`, `sonar`, `heart`: $n_2 < 75$) all certificates are vacuous or nearly so: with 54 examples, $\ln(2\sqrt{n}/\delta)/n \approx 0.09$ even with KL $= 0$, and the factor 2 or 4 does the rest. On the larger datasets (`spambase`, `pageblocks`, `satimage`, `segmentation`) they become informative, e.g. 0.124 for `pageblocks` whose vote errs on 2.1% of the test points.
Which bound is the tightest? Proposition 5.7 says that, as *oracle* quantities, the C-bound and the tandem bound beat the first-order bound when $d \geq R(G)$. This condition holds on almost all datasets (compare the columns Gibbs and d), but the certificates tell a different story: the first-order certificate is the smallest on every dataset except `spambase` (on `bupa` and `sonar` all three are vacuous). The reason is statistical: the tandem bound is estimated on $n_2 \approx 0.55\, n_1$ examples and multiplied by 4, and the C-bound needs two kl inversions. Only when the sample is large and the trees diverse enough (`spambase`: $d = 0.156 > R = 0.124$, $n_2 = 1119$) does the tandem certificate win. The best oracle bound is not necessarily the best certificate.
3. The two forests have very close test risks (e.g. 0.051 vs 0.056 on `spambase`, 0.137 vs 0.141 on `australian`); on some datasets `BaggedTrees` is even slightly better. Halving the bootstrap samples costs little accuracy, while it raises the OOB proportions from $e^{-1} \approx 37\%$ to $e^{-1/2} \approx 61\%$ (and from 14% to 37% for pairs), which makes the certificates much tighter. Part 1.4 studies this trade-off.

Part 1.4 – Number of trees and size of the bootstrap samples (Section 7.2)

On `spambase` (3 splits), learn forests with $n \in \{10, 50, 100, 200\}$ trees and bootstrap samples of size $k = f\, m$ with $f \in \{0.3, 0.5, 0.8, 1\}$.

1. Before running anything, predict how n_1 and n_2 depend on f (compute $(1 - 1/m)^{fm}$ and its square), and whether they depend on n.

2. Report the test risk of the uniform vote, n_1, n_2 and the FO and TND certificates. Which configuration gives the best vote? the best certificate? Is there a configuration that is good for both?

Solution

1. An example is out of a bootstrap sample of size fm with probability $(1 - \frac{1}{m})^{fm} \approx e^{-f}$, and out of two independent samples with probability $\approx e^{-2f}$. With $m = 3217$ training examples, $n_1 \approx e^{-f} m$ gives $2383, 1951, 1446, 1183$ and $n_2 \approx e^{-2f} m$ gives $1765, 1183, 650, 435$ for $f = 0.3, 0.5, 0.8, 1$; the minima over trees and pairs are a little smaller. The expected sizes do not depend on n, but the minima decrease slowly with n (a minimum over more

trees or pairs).

Algorithm 4: Grid over the number of trees and the bootstrap size

```
for f ∈ {0.3, 0.5, 0.8, 1}, n ∈ {10, 50, 100, 200}, split s = 1, 2, 3 do
    learn BaggedTrees(n, f) on the training part; compute the OOB statistics
      (Part 1.3);
    record test risk of the uniform vote, n_1, n_2, FO and TND certificates;
average over the splits;
```

Python: function `partA` of `tp_solutions.py`.

f	n	test risk	n_1	n_2	FO cert.	TND cert.
0.3	10	0.078	2370	1734	0.334	0.331
0.3	50	0.058	2361	1708	0.326	0.292
0.3	100	0.057	2358	1706	0.327	0.286
0.3	200	0.057	2358	1706	0.328	0.284
0.5	10	0.071	1931	1146	0.310	0.323
0.5	50	0.054	1922	1128	0.307	0.293
0.5	100	0.055	1918	1117	0.308	0.291
0.5	200	0.054	1918	1109	0.308	0.287
0.8	10	0.065	1422	615	0.300	0.354
0.8	50	0.053	1408	586	0.296	0.322
0.8	100	0.049	1406	584	0.299	0.321
0.8	200	0.049	1402	577	0.300	0.319
1.0	10	0.067	1141	392	0.291	0.393
1.0	50	0.050	1136	380	0.299	0.362
1.0	100	0.048	1136	371	0.296	0.355
1.0	200	0.050	1135	370	0.297	0.356

2. The measured n_1, n_2 match the prediction. *Number of trees:* going from 10 to 50 trees improves the vote ($0.078 \to 0.058$ for $f = 0.3$) and the tandem certificate ($0.331 \to 0.292$), because the joint errors of more trees average out; beyond 100 trees nothing changes. *Bootstrap size:* larger bootstrap samples give better trees and a slightly better vote (0.057 for $f = 0.3$, 0.048 for $f = 1$ with 100 trees) and a slightly smaller first-order certificate, but they shrink the pairwise OOB sets from 1706 to 371 examples, and the tandem certificate degrades from 0.286 to 0.355. There is no free lunch: the best vote ($f = 1$) and the best certificate ($f = 0.3$, 200 trees: 0.284) are obtained for different configurations. $f = 0.5$ with 100 trees is a reasonable compromise (0.055 / 0.291), which is why it is the default of the course. Note that choosing the configuration *after* seeing the certificates would require a union bound over the 16 configurations ($\ln 16 \approx 2.8$ nats).